Agentic AI for Rockburst Intensity Assessment

Client brief review: task breakdown and required tech stack

1. What the client wants

The client wants a **physics-guided, multi-agent AI framework** that predicts rockburst intensity (grade 0–3) and also judges how much the prediction can be trusted. Accuracy alone is explicitly not the goal.

The system must be **truly agentic**. A central planner looks at the state of each case and decides which agent or tool to call next. A fixed pipeline that runs the same six agents on every sample is not acceptable.

The deliverable is a working, reproducible implementation, a methodology document and a short technical report. The brief is written like an acceptance-test document, and section 10 of it lists the traps they will check for.

2. The dataset (from the PDF)

- **Size and features:** 344 cases with 8 inputs: σθ (tangential stress), σc (compressive strength), σt (tensile strength), SCF, B1, B2, Wet (elastic energy index) and D (depth, m). The target is the rockburst grade 0–3.
- **The PDF is only a table, not a CSV.** You will have to parse it, and the text extraction has damage. Several rows are fused together (rows 38/39, 81/82, 124/125, 210/211, 253/254, 296/297, 339/340), so verify the parsing against the row numbers.
- **D is missing in a large share of rows** (NaN). Some are consecutive blocks from a single source, so the missingness is not random. This is the client's "missing-data challenge".
- **Some features are exact derived quantities.** Checked on a few rows: SCF = σθ/σc (row 1: 90/170 = 0.53); B1 = σc/σt (170/11.3 = 15.04); B2 = (σc−σt)/(σc+σt) (0.88). This is what the Geomechanical Agent should verify deterministically. It should treat these as exact relationships and Wet and D as empirical tendencies.
- **The data contains real errors** the Data Quality and Physics agents should catch: rows 20–21 have B1 = 0.15 where the formula gives about 14.7; row 244 lists B1 = 21.2 where 125.37/7.74 = 16.2; rows 146–157 have σθ of 225–297 MPa (SCF above 2), which is out-of-distribution; and there are exact and near-duplicates (rows 187 and 180 are identical, rows 192 and 165 are identical). Many cases also recur across the literature sources, for example rows 1 and 327, or 2 and 324, with slightly different grades.
- **Duplicates make leakage the main ML risk.** A random split would put near-identical cases in train and test. Use grouped or de-duplicated splits.
- **The PDF also contains PCA and factor-loading tables (S2–S4).** They are background from the source paper. The client doesn't ask you to reproduce them.

3. Tasks in detail

A. Design documents

- An architecture diagram and a written methodology.
- A spec for each agent (role, inputs, outputs, tools).

- The state-transition design, with a max loop count and explicit termination and abstention rules.
- The mathematical formulation of the reliability fusion, R_final = f(R_data, R_physics, R_model, R_uncertainty, R_stability).
- The leakage-safe validation design, the missing-data strategy, and the uncertainty and abstention method.

B. The six agents (each backed by real computation, not prompts alone)

Agent	What it must do	Deterministic tools
Coordinator / Planner	Holds the case state, picks the next action, handles loops, stops or abstains	State machine or graph, routing rules
Data Quality & Anomaly	Checks missingness, outliers, impossible combinations, near-duplicates, scale issues and OOD. Returns **R_data ∈ [0,1]**	Isolation Forest or Mahalanobis distance, IQR/z-scores, kNN distance, range rules
Geomechanical Reasoning	Verifies SCF, B1 and B2 formulas, and checks that Wet and D fall in plausible ranges. Separates exact from empirical relations. Returns **R_physics**	Python functions with tolerances, no LLM arithmetic
Predictive Modelling	Trains and compares logistic/ordinal regression, SVM, Random Forest, XGBoost/LightGBM/CatBoost, plus one more justified tabular method	scikit-learn pipelines, an ordinal model such as mord or CORAL-style, nested CV
Uncertainty & Reliability	Calibrated probabilities, model disagreement, perturbation sensitivity, prediction sets, abstention	Calibration, conformal prediction, ensembles, Monte-Carlo perturbation
Critic / Verification	Independently challenges the result, finds contradictions across agents, and can send the case back for a physics, data or model re-check	Rule-based contradiction checks, an independent model or perturbation test, optionally an LLM to reason over structured outputs

C. Dynamic routing

The loop is State → Action → Observation → Updated State → Next Decision. Examples of required conditional behaviour:

- Missing or suspicious inputs trigger extra validation.
- Strong model disagreement triggers the Critic.
- The Critic can request another check.
- There is a hard loop limit, and the controller can abstain.

D. Technical requirements

- **Leakage Guard.** Imputation, scaling, feature selection, oversampling and tuning are fitted on training folds only. Observed and imputed features are marked, and confidence drops when important features are reconstructed. The target is never used for imputation.
- **Missing-data experiments.** Compare several imputers, not just the mean. Evaluate complete cases, one key feature missing, multiple features missing, and training-time versus inference-time missingness.
- **Ordinal treatment.** Compare plain multiclass with ordinal models. Metrics must penalise a large error ($|\hat{y} - y|$) more than an adjacent-grade error.

- **Robustness.** Perturb inputs with $x' = x(1+\varepsilon)$ at a mild and a strong range. Measure class flips, probability shifts and uncertainty before and after. The system must become less confident as evidence degrades.
- **Fusion.** No majority voting and no hand-picked weights. The weights must be learned, calibrated or optimised (for example a logistic stacker on out-of-fold data, or a constrained optimisation) and then justified.
- **Structured tool use.** Agents exchange JSON messages. The LLM orchestrates and explains, while Python does all the numbers.

E. Experiments and evaluation

- **Baselines first (M1).**
- **Mandatory ablation, M1–M7:** M1 traditional ML; M2 ML + explainability; M3 ML + physics; M4 ML + uncertainty; M5 single agent; M6 multi-agent without Critic; M7 full system.
- **Metrics:** accuracy, balanced accuracy, macro-F1, class-wise recall, confusion matrix; quadratic weighted kappa and ordinal MAE; Brier score and ECE; prediction-set coverage and size; severe misclassification rate; abstention rate and risk-coverage curve; robustness under perturbation; agent contradiction-detection rate.

F. Outputs and evidence

- **Per-case engineering assessment.** The fields are predicted grade, probability, prediction set, physics consistency, input reliability, OOD status, stability, agent disagreement, final confidence and a plain-language decision.
- **Evidence the system runs:** orchestration and tool source code; a notebook or script that runs from the raw dataset to the final output; saved model artefacts or exact reproducible training instructions; at least three example JSON outputs; a trace where routing changes because of missingness, OOD, disagreement or Critic feedback; one abstention example; one failure analysis; seeds, config files and an environment spec; a short technical report with limitations.
- **Honesty requirement.** Report cases where the full agentic system does worse than a baseline. Do not claim that more agents automatically means better.

4. Recommended tech stack

Layer	Suggested tools
Language and environment	Python 3.10+, requirements.txt or environment.yml (or Poetry), fixed seeds, and DVC or a simple config-driven run for reproducibility
Data extraction	pdfplumber or camelot (with manual verification) to get the table from the PDF, then pandas and numpy
Classical ML	scikit-learn (pipelines, SVM, RF, logistic, calibration), xgboost, lightgbm, catboost. Add TabPFN or an ExtraTrees/TabNet-type model as the extra tabular method
Ordinal modelling	mord, statsmodels OrderedModel, a CORAL/CORN-style implementation in PyTorch, or an ordinal-decomposition wrapper around tree models
Imputation and leakage safety	IterativeImputer, KNNImputer, missing-indicator features and MICE-style methods, all inside sklearn.pipeline. Use StratifiedGroupKFold with nested CV, and Optuna for tuning inside the training folds
Imbalance handling	imbalanced-learn (SMOTE only inside the training fold) or class weights
Anomaly and OOD	IsolationForest, LocalOutlierFactor, Mahalanobis distance, kNN distance (scikit-learn / scipy)

Layer	Suggested tools
Uncertainty	Temperature or isotonic calibration, MAPIE or a hand-written split or Mondrian conformal predictor, bootstrap ensembles, entropy and disagreement metrics
Explainability (M2)	shap, permutation importance, partial dependence
Physics module	Plain Python and numpy/scipy functions with tolerance rules and rock-mechanics references
Agent orchestration	**LangGraph** is the natural fit for the state-based routing; a custom FSM would also satisfy the brief. Use pydantic for the JSON message schemas
LLM layer	Claude (or another LLM) through the API with tool/function calling. The LLM plans and explains, while every number comes from Python tools
Fusion	Logistic-regression stacking or constrained optimisation with scipy.optimize (weights learned on out-of-fold data)
Evaluation and plots	scikit-learn.metrics, matplotlib/seaborn, custom ECE, QWK, risk-coverage and reliability diagrams
Tracking and logging	MLflow or Weights & Biases, plus JSON trace logs for the routing evidence
Deliverables	Jupyter notebooks, a Makefile or CLI script, pytest for the tools, Mermaid or draw.io for the architecture diagram, and a Word/PDF or Markdown report